UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Experiment 3

Student Name: Prateek Verma | **UID: 25MCC20062**
Branch: MCA (CC & DEVOPS) | **Section/Group: 25MCC-101/A**
Semester: IInd | **Date of Performance: 27/01/2026**
Subject Name: Technical Training | **Subject Code: 25CAP-652**

Title: Implementation of Conditional Logic using IF–ELSE and CASE Statements in PostgreSQL

1. **Aim:**
   a. To implement conditional decision-making logic in PostgreSQL using IF–ELSE constructs and CASE expressions for classification, validation, and rule-based data processing.

2. **Software Requirements**:
   a. PostgreSQL (Database Server)
   b. pgAdmin
   c. OS: Windows

3. **Objective:**
   a. To understand conditional execution in SQL
   b. To implement decision-making logic using CASE expressions
   c. To simulate real-world rule validation scenarios
   d. To classify data based on multiple conditions
   e. To strengthen SQL logic skills required in interviews and backend systems

4. **Procedure of the Practical:**
   a. Start the system.
   b. Open pgAdmin.
   c. Create and select the database in which you want to perform the experiment.
   d. Establish connection to the database using Alt+Shift+Q.
   e. Run the following queries given in Experiment Steps (5).

5. Experiment Steps:

a. Create table queries:

i. Create table schema_violations (schema_id int primary key, schema_name varchar(50) not null, violation_count int not null check (violation_count >= 0));

b. Insert queries:

i. Insert into schema_violations values (1,'finance_schema', 0);

	schema_id [PK] integer	schema_name character varying (50)	violation_count integer
1	1	Finance_Schema	0
2	2	HR_Schema	2
3	3	Sales_Schema	5
4	4	Inventory_Schema	9
5	5	Audit_Schema	12
6	6	Compliance_Schema	20

c. Classifying data using case expression:

i. Select *, case
when violation_count = 0 then 'no violation'
when violation_count between 1 and 3 then 'minor violation'
when violation_count between 4 and 7 then 'moderate violation'
else 'critical violation'
end as violation_category
from schema_violations;

	schema_id [PK] integer	schema_name character varying (50)	violation_count integer	violation_category text
1	1	Finance_Schema	0	No Violation
2	2	HR_Schema	2	Minor Violation
3	3	Sales_Schema	5	Moderate Violation
4	4	Inventory_Schema	9	Critical Violation
5	5	Audit_Schema	12	Critical Violation
6	6	Compliance_Schema	20	Critical Violation

d. Applying case logic in data updates:

i. Alter table to add status attribute:
Alter table schema_violations add column approval_status varchar(20);

ii. Updating status using case logic:
Update schema_violations set approval_status = case when violation_count = 0 then 'Approved' when violation_count between 1 and 7 then 'Needs Review' else 'Rejected' end;

	schema_id [PK] integer	schema_name character varying (50)	violation_count integer	approval_status character varying (20)
1	1	Finance_Schema	0	Approved
2	2	HR_Schema	2	Needs Review
3	3	Sales_Schema	5	Needs Review
4	4	Inventory_Schema	9	Rejected
5	5	Audit_Schema	12	Rejected
6	6	Compliance_Schema	20	Rejected

e. Using case statement for custom sorting (descending):

i. Select * from schema_violations order by case when approval_status='rejected' then 1 when approval_status='needs review' then 2 when approval_status='approved' then 3 end;

	schema_id [PK] integer	schema_name character varying (50)	violation_count integer	approval_status character varying (20)
1	4	Inventory_Schema	9	Rejected
2	5	Audit_Schema	12	Rejected
3	6	Compliance_Schema	20	Rejected
4	2	HR_Schema	2	Needs Review
5	3	Sales_Schema	5	Needs Review
6	1	Finance_Schema	0	Approved

f. Real world classification scenario (grading system):

i. Create table:

- create table students (id int primary key, name varchar(30) not null, marks int not null check(marks<>0 and marks<=100));

ii. Inserting records:

- Insert into students (id, name, marks) values (1, 'Aarav', 95);

	id [PK] integer	name character varying (30)	marks integer
1	1	Aarav	95
2	2	Meera	82
3	3	Rohit	68
4	4	Sneha	54
5	5	Kunal	40
6	6	Priya	25

iii. Adding new grade column:

- alter table students add column grade varchar(2);

iv. Updating grade column to reflect student grade based on marks using case statement:

- Update students set grade = case when marks >= 90 then 'A+' when marks >= 80 then 'A' when marks >= 70 then 'B' when marks >= 60 then 'C' when marks >= 50 then 'D' else 'F' end;

	id [PK] integer	name character varying (30)	marks integer	grade character varying (2)
1	1	Aarav	95	A+
2	2	Meera	82	A
3	3	Rohit	68	C
4	4	Sneha	54	D
5	5	Kunal	40	F
6	6	Priya	25	F

g. PL/pgSQL block to implement IF-ELSE logic

i.
```
do $$
declare
   i int := 7;
begin
   if i = 0 then
      raise notice 'no violation';
   elsif i <= 3 then
      raise notice 'minor violation';
   elsif i <= 7 then
      raise notice 'moderate violation';
   else
      raise notice 'critical violation';
   end if;
end $$;
```

```
NOTICE:  MODERATE VIOLATION
DO

Query returned successfully in 42 msec.
```

6. Input/Output Details:

a. The input for the experiment is the SQL queries mentioned in Experiment Steps (5).

b. The output for each query is attached after the query itself.

7. Learning Outcome

a. Knowledge on how conditional logic is implemented in PostgreSQL using CASE expressions and IF–ELSE constructs.
b. Strong command over rule-based SQL logic, which is essential for:
    i. Backend systems
    ii. Analytics
    iii. Compliance reporting
    iv. Placement and technical interviews